

Lab 4

Solution

2025-06-29

Short Answer Question

Question 1

Using this source, describe the file drawer problem in your own words.

The file drawer problem is when results that are not statistically significant or boring results are not published simply “filed away”. This leads to a bias in research since we only see studies with interesting and significant results without seeing what kind of statistics in that realm was discarded making that positive statistic seem supported by science just because the discarded information was not shared.

Question 2

What is the duality between a confidence interval and its related hypothesis test?

If the value stated in the null hypothesis is inside the confidence interval, you fail to reject the null. If it is outside you reject the null hypothesis.

Question 3

How do we prevent overfitting?

Overfitting is prevented by using a train/test split on the model, using more data, and validating performance on unseen data through methods like cross-validation.

Question 4

Describe the bias-variance trade-off.

The greater the bias the lesser the variance because it would only lean to one direction from the bias.

Coding Question

Section 1: Set Up

Load tidyverse.

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.4
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Set your seed to a value of your choosing.

```
set.seed(100)
```

Load the package kmed.

```
library(kmed)
```

```
## Warning: package 'kmed' was built under R version 4.5.1
```

Load the data heart.

```
data(heart)
```

Load the package MASS.

```
library(MASS)
```

```
## Warning: package 'MASS' was built under R version 4.5.1
```

```
##
```

```
## Attaching package: 'MASS'
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

```
##      select
```

Load the data crabs.

```
data(crabs)
```

Section 2: Duality of Hypothesis Test and Confidence Intervals

We are going to test the difference in frontal lobe size between the sexes.

Find the mean frontal lobe size for male crabs and female crabs.

```
crabs |>
  group_by(sex) |>
  summarize(mean = mean(FL))
```

```
## # A tibble: 2 x 2
##   sex      mean
##   <fct> <dbl>
## 1 F      15.4
## 2 M      15.7
```

Task 1: Create a 95% confidence interval using the following equation:

$$(\bar{x}_{female} - \bar{x}_{male}) \pm t_{\frac{\alpha}{2}, df} SE$$

First, define the following values and create values to save these values in R: (remember that $df = n_1 + n_2 - 2$ and $SE = \sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}$).

$\bar{x}_{male} =$

$\bar{x}_{female} =$

$\alpha =$

$df =$

$SE =$

```
mean_male = mean(crabs$FL[crabs$sex == "M"])
#print(mean_male)
mean_female = mean(crabs$FL[crabs$sex == "F"])
#print(mean_female)

n1 = sum(crabs$sex == "M")
n2 = sum(crabs$sex == "F")

df = n1 + n2 - 2
#print(df)

sd1 <- crabs |>
  filter(sex == "F") |>
  summarize(sd_f = sd(FL)) |>
  pull()

sd2 <- crabs |>
  filter(sex == "M") |>
  summarize(sd_m = sd(FL)) |>
  pull()

SE = sqrt(((sd1)^2/n1) + ((sd2)^2/n2))
#SE
qt = qt(0.975, df = df)
```

Now, determine the lower and upper bounds of the confidence interval.

Hint: use qt()

```
diff = mean_female - mean_male
upper = diff + qt * SE
print(upper)
```

```
## [1] 0.6743367
```

```
lower = diff - qt * SE
print(lower)
```

```
## [1] -1.278337
```

Does this confidence interval include zero? What does this mean?

This means there is a possibility that the male front lobe sizes could be equal to the female frontal lobe sizes.

Task 2: Run a two-sided hypothesis test to test if male vs. female crabs have different frontal lobe sizes.

Write out your null and alternative hypothesis:

H_0 : Both the male and female crabs have the same frontal lobe sizes.

H_A : The male and female crabs have different frontal lobe sizes.

Determine your observed t-statistic. (Some of the values from Task 1 will be helpful here.)

```
obs_t <- t.test(FL~sex, data = crabs)$statistic
obs_t
```

```
##          t
## -0.6099835
```

Use bootstrap to create your null distribution (500 reps) and plot it with a vertical line at your t-statistic.

```
bootstrap_t <- function(data) {
  data$sex <- sample(data$sex)
  t.test(FL~sex, data = data)$statistic
}
bs_t <- replicate(500, bootstrap_t(crabs))
```

Use quantile() to determine the rejection cutoffs.

```
cutoff <- quantile(bs_t, c(.025, .975))
```

Use between() to determine if the observed t statistic is statistically significant.

```
between(x = obs_t, left = cutoff[1], right = cutoff[2])
```

```
## [1] TRUE
```

What are the conclusions of this hypothesis test?

We conclude that there is not enough evidence to suggest a significant difference in FL by sex. The observed t-statistic falls within the expected range under the null hypothesis, so we fail to reject the null.

How do these compare to the results from your confidence test?

The observed t statistic was within the confidence interval.

Task 3: Run an analytical version of the t-test using `t.test`. Compare the results to your confidence interval and simulated hypothesis test.

This also shows that the observed t statistic is within the confidence interval.

```
t.test(FL~sex, data=crabs)

##
## Welch Two Sample t-test
##
## data: FL by sex
## t = -0.60998, df = 197.91, p-value = 0.5426
## alternative hypothesis: true difference in means between group F and group M is not equal to 0
## 95 percent confidence interval:
## -1.2783394 0.6743394
## sample estimates:
## mean in group F mean in group M
## 15.432 15.734
```

Section 3: Testing and Training with Linear Regression

We are going to fit a simple linear model of the crabs dataset with several different degrees, and compare the R^2 statistic across the training and testing tests in order to determine model accuracy.

First, create a vector of 80% “train” and 20% “test” that is the same length as the crabs dataset.

Hint: use the `prob` argument in `sample()`.

```
split <- sample(c("train", "test"),
               size = nrow(crabs),
               replace = TRUE,
               prob = c(0.8, 0.2))
```

Add this vector as a new column into the crabs dataset and save it back into itself.

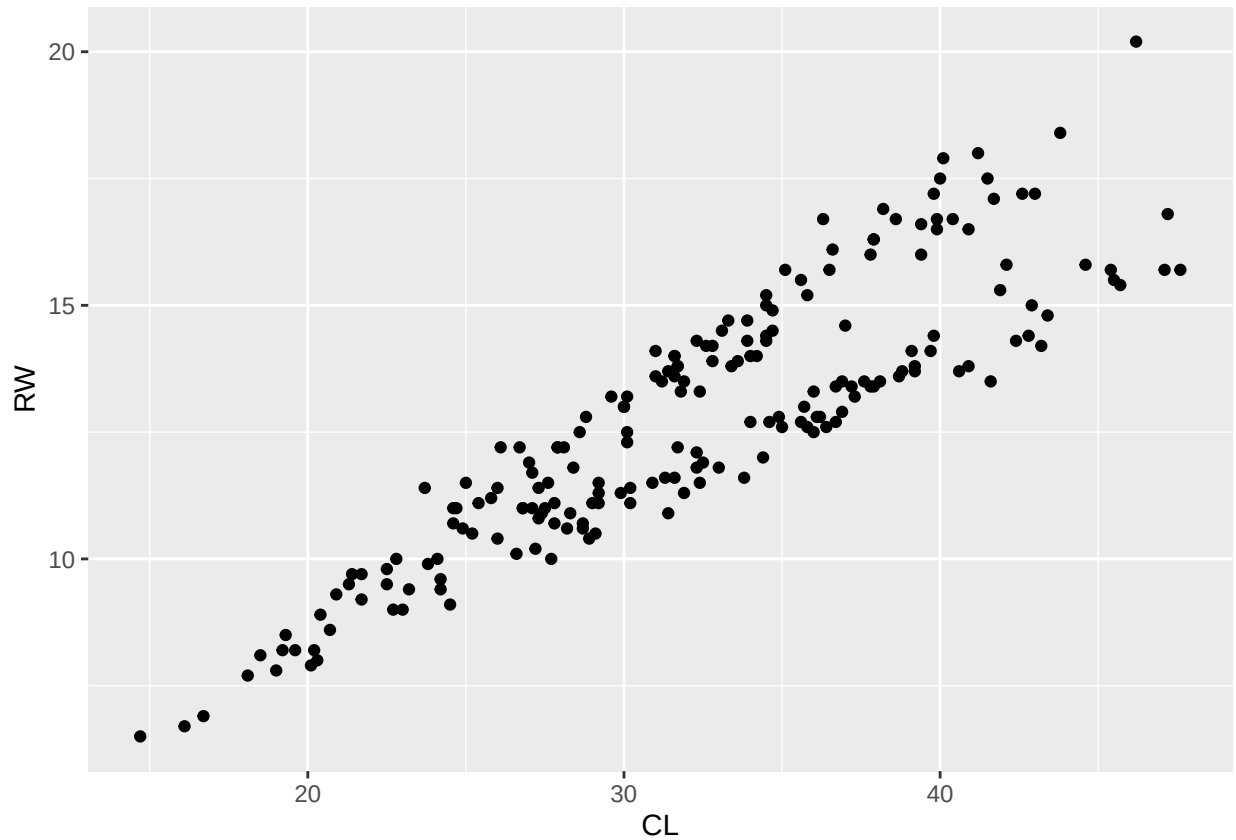
```
crabs <- crabs |>
  mutate(split = split)
```

Use this column to create a train dataframe and a test dataset, and name them as such.

```
train <- crabs |>
  filter(split == "train")
test <- crabs |>
  filter(split == "test")
```

Plot carapace length on the x and rear width on the y.

```
crabs |>
  ggplot(aes(x = CL, y=RW))+
  geom_point()
```



Create a linear model called `m1` that uses carapace length to predict rear width. Remember to use the training dataset to create this model.

```
m1 <- lm(RW ~ CL, data = train)
```

Use `predict()` to create a `predicted_value` column in the training dataset. Make sure to use the `new_data` argument.

```
train <- train |>
  mutate(predicted_value = predict(m1, newdata = train))
```

Repeat above for the testing dataset.

```
test <- test |>
  mutate(predicted_value = predict(m1, newdata = test))
```

Write a function that calculates the RSS.

```
RSE <- function(data){
  RSS <- sum((data$R - data$predicted_val)^2)
  n <- nrow(data)
  sqrt(RSS/(n-2))
}
```

What is the RSS for the training and testing dataset?

```
RSE(train)
```

```
## [1] 1.208431
```

```
RSE(test)
```

```
## [1] 0.9829468
```

Do you think there are any signs of overfitting?

It seems like there is a big gap between the accuracies of the training and testing dataset so I do not think that the testing dataset had adapted well with the training data algorithm so there are signs of overfitting.

Fit a second model called `m_10` with the same variables but use a polynomial of degree 15 using the `poly()` function. Remember to use the training dataset.

```
m_10 <- lm(RW ~ poly(CL, degree = 15), data=train)
```

Repeat the steps above to find the RSE of this new model.

```
train <- train |>
  mutate(predicted_value = predict(m_10, newdata = train))
test <- test |>
  mutate(predicted_value = predict(m_10, newdata = test))

RSE(train)
```

```
## [1] 1.150223
```

```
RSE(test)
```

```
## [1] 3.086176
```

Do you think there are any signs of overfitting?

This high degree polynomial fits the training data extremely closely, even capturing noise but fail to generalize to new data, leading to large test error.

Section 4: ANOVA Test

We are now going to be working with the heart dataset.

In this dataset, there is a variable called `cp`. What does this stand for? And what are the different levels of this variable?

Four chest pain types: (1) typical angina, (2) atypical angina (3)non-anginal pain, (4) asymptomatic (categorical).

Create a new column called `cp_words` that converts the numeric column into labels that are more user-friendly. Save this column back into the heart dataset.

Hint: use `case_when`.

```
heart <- heart |>
  mutate(cp_words = case_when(
    cp == 1 ~ "typical angina",
    cp == 2 ~ "atypica angina",
    cp == 3 ~ "non-anginal pain",
    cp == 4 ~ "asymptomatic"
  ))
```

Use `aov()` to get the ANOVA table with `chol` as the response variable and `cp_words` as the factor variable. Save this as an object called `anova`.

```
anova <- aov(chol ~ cp_words, data=heart)
```

Use `summary()` on the `anova` object from above. Do you get a statistically significant result?

```
summary(anova)
```

```
##              Df Sum Sq Mean Sq F value Pr(>F)
## cp_words      3   4687    1562   0.575  0.632
## Residuals    293  795623    2715
```

Interpret these results.

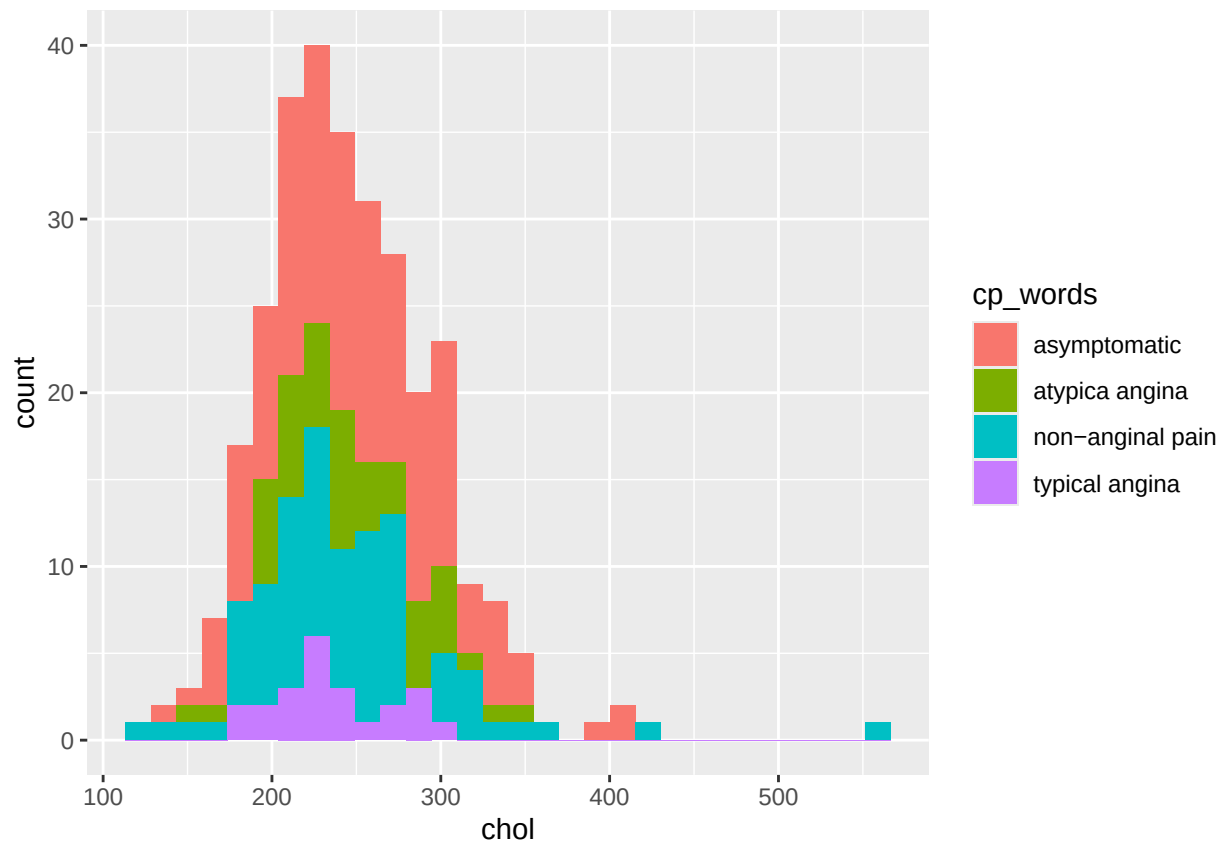
No, this result is not statistically significant because the p-value (0.632) is much greater than 0.05. We fail to reject the null hypothesis that the group means of `cp` are equal.

Plot `chol` with fill representing `cp_words`. Do you agree with this conclusion?

Yes, I agree with this conclusion because it shows the same distribution as was shown from the summary of the ANOVA test.

```
heart |>
  ggplot(aes(x = chol, fill = cp_words))+
  geom_histogram()
```

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```

Let's try using bootstrap to run this same hypothesis test.

Get the observed test statistic (in this case, an F-statistic using `aov()`). The syntax can be clunky.

```
summary(anova)[[1]]$'F value'[[1]]
```

```
## [1] 0.5753347
```

Create a bootstrap function that permutes the factoring variable and takes the necessary test.

```
bootstrap <- function(data){
  anova <- aov(chol ~ cp_words, data = data)
  summary(anova)[[1]]$'F value'[[1]]
}
bootstrap(heart)
```

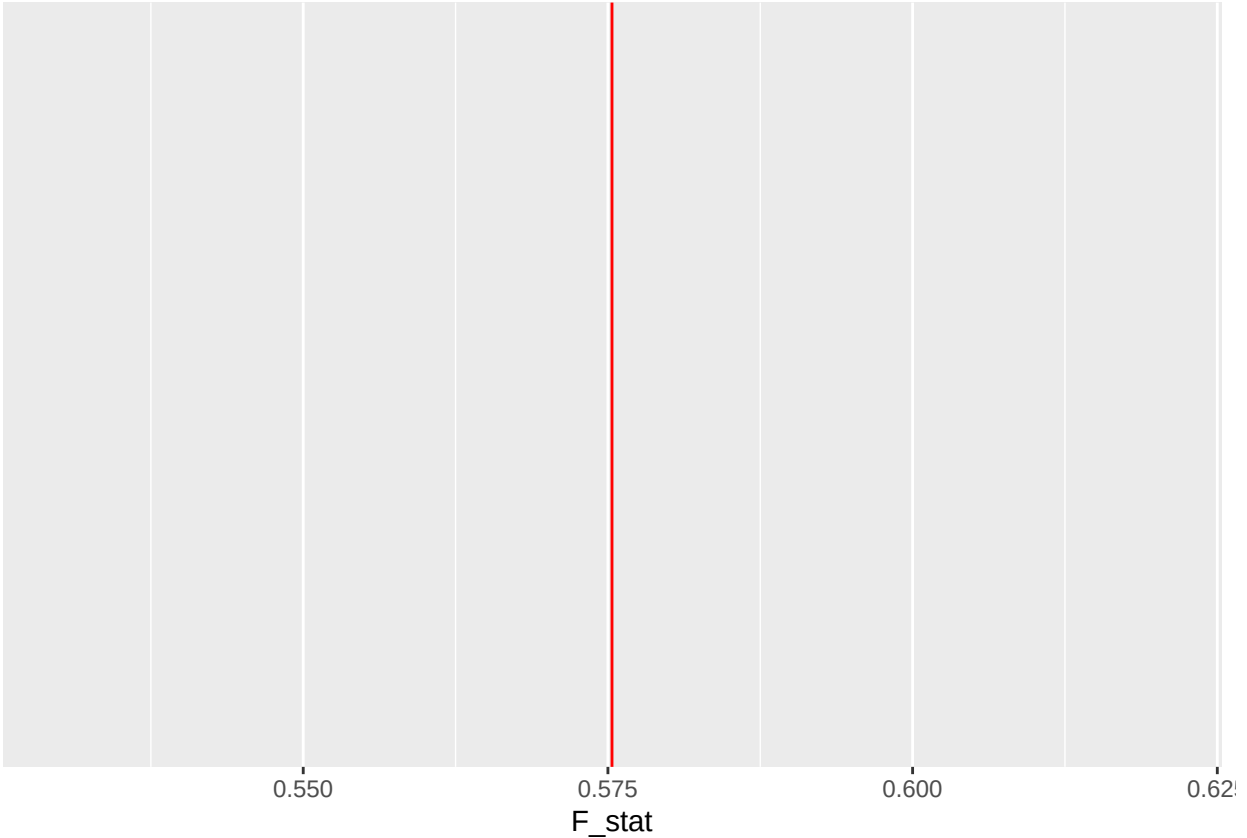
```
## [1] 0.5753347
```

Replicate this bootstrap function 500 times.

```
big_data <- replicate(500, bootstrap(heart))
```

Plot these bootstrap F-statistics with a vertical line for the observed test statistics.

```
obs_f <- summary(aov(chol ~ cp_words, data = heart))[[1]]$'F value'[[1]]
f_data <- data.frame(F_stat = big_data)
ggplot(f_data, aes(x = F_stat)) +
  geom_vline(xintercept = obs_f, color = "red")
```



Does this visually look statistically significant?

No because it is just a line and it is not showing any significance since it is not relative to a broader distribution.

How does the F-distribution look different from the other bootstrap distributions?

Bootstrap has a histogram and a line where the mean and sd are kind of at or where the confidence interval is bounded so it makes more sense.

Use `quantile()` to find the rejection threshold for a 5% significance level.

```
rejection_cutoff <- quantile(big_data, 0.95)
```

Is our observed test statistic statistically significant?

```
obs_f > rejection_cutoff
```

```
## 95%
## FALSE
```

Interpret the results.

This observed statistic was not within the 95% confidence interval.

Section 5: Chi-Squared Test

We are back with functions and fake data!

Task 1: Test 10 tosses of a slightly unfair coin.

Create 10 draws from a binomial distribution where $n = 1$ and $p = 0.6$. Let this represent 10 tosses of a slightly unfair coin.

```
fake_coin <- rbinom(10, 1, 0.6)
```

Write a function that returns a vector of length 2: the first number is how many heads (or 0s) and the second number if how many tails (or 1s). Call this function `counting_freq`

```
counting_freq <- function(tosses){  
  c(sum(tosses == 0), sum(tosses ==1))  
}
```

Test this function on your fake dataset.

```
counting_freq(fake_coin)
```

```
## [1] 4 6
```

Run a chi-squared test on the frequencies using `chisq.test()` .

```
chisq.test(x=counting_freq(fake_coin))
```

```
##  
## Chi-squared test for given probabilities  
##  
## data:  counting_freq(fake_coin)  
## X-squared = 0.4, df = 1, p-value = 0.5271
```

Interpret these results including the null and alternative hypothesis and whether or not these results are statistically significant.

Task 2: Repeat the above with 1000 tosses of a coin. Do you get a statistically significant result?

```
fake_coin <- rbinom(1000, 1, 0.6)  
chisq.test(x=counting_freq(fake_coin))
```

```
##  
## Chi-squared test for given probabilities  
##  
## data:  counting_freq(fake_coin)  
## X-squared = 33.124, df = 1, p-value = 8.646e-09
```

Section 6: KNN

Task 1: Make sure that `cp_words` is a factor!

```
heart <- heart |>
  mutate(cp_words = as.factor(cp_words))
```

Task 2: Create a 80/20 training/testing split with the heart data using indices.

```
# heart$cp_words <- as.factor(heart$cp_words)
#
# n <- nrow(heart)
# train_idx <- sample(1:n, 0.8 * n)
# train <- heart[train_idx, ]
# test <- heart[-train_idx, ]
#
# fit <- knn3(cp_words ~ ., data = train, k = 1)
#
# train_acc <- mean(predict(fit, train, type = "class") == train$cp_words)
# test_acc <- mean(predict(fit, test, type = "class") == test$cp_words)
#
# train_acc
# test_acc
```

Task 2: Load library caret .

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.5.1
```

```
## Loading required package: lattice
```

```
##
```

```
## Attaching package: 'caret'
```

```
## The following object is masked from 'package:purrr':
```

```
##
```

```
## lift
```

Task 3: Create a KNN model using the training dataset to predict cp_words and k = 1. Find the training and testing accuracy.

The syntax is different from what we've worked with so far, so we will work through this together in class.

```
# knn_fit1 <- knn3(cp_words ~ ., k=1, data=train)
# train|>
#   mutate(expected = predict(knn_fit1, newdata = train, type = 'class'),
#     correct = cp_words == expected)|>
#   summarize(accuracy = mean(correct))
# test|>
#   mutate(expected = predict(knn_fit1, newdata = test, type = 'class'),
#     correct = cp_words == expected)|>
#   summarize(accuracy = mean(correct))
```

How do the models compare?

In the second model our k value had increased to 0.4 while the first model had given us a k value of 0.3166666. This increase in value led to a greater accuracy since

Task 4: Repeat task 4 with $k = 1, \dots, 40$. (Use a for-loop to optimize this process). Create a plot of the training and testing accuracy rates vs. k.

```
# results <- data.frame(k = 1:40, train_acc = NA, test_acc = NA)
# for (k in 1:40) {
#   model <- knn3(cp_words ~ ., data = train, k = k)
#   train_pred <- predict(model, train, type = "class")
#   test_pred <- predict(model, test, type = "class")
#
#   results$train_acc[k] <- mean(train_pred == train$cp_words)
#   results$test_acc[k] <- mean(test_pred == test$cp_words)
# }
# ggplot(results, aes(x = k)) +
#   geom_line(aes(y = train_acc, color = "Train")) +
#   geom_line(aes(y = test_acc, color = "Test"))
```

What do you think is the ideal k value for this dataset? Why?

The ideal k value is around 20 since that is when the graphs coincide and have an equal training and testing accuracy (so it is not prone to overfitting).